

A Blockchain-based, Semantics-driven, Modular Implementation of BPMN Choreographies

– Technical report –

Nawaz Abdullah Malla¹, Alessandro Marcelletti¹, Andrea Morichetta¹, and
Francesco Tiezzi²

¹ University of Camerino, Italy

² University of Florence, Italy

1 BPMN Choreography Formalization

This section presents the formalization of BPMN choreographies we rely on. This formalization results from the extension of the one presented in [1] with additional features inspired by the formalization in [2].

The graphical notation of BPMN choreographies makes unhandy the definition of the formal semantics. Therefore, the formalization is based on an alternative textual representation of choreographies, whose syntax is defined by the BNF grammar in Fig. 1. In the grammar, the non-terminal symbol C represents *Choreography Structures*, while the terminal symbols, denoted by the **sans serif** font, are the considered elements of a BPMN model, i.e. events, gateways and tasks. Non-terminal symbols X and T represent expression lists and event-based task lists, respectively; when convenient, we shall regard these lists simply as sets. As a matter of notation, let $\mathbb{E}$ be the set of *sequence edge* names, in the following $e \in \mathbb{E}$ denotes an edge, while $E \in 2^{\mathbb{E}}$ a set of edges. Notice that the set $\mathbb{E}$ also includes spurious edges for denoting the enabled/disabled status of start events. Moreover, the set $\mathbb{M}$ of *messages* is ranged over by m , while the set $\mathbb{V}$ of *values* is ranged over by v . Notation p ranges over names uniquely identifying *participants*, while id ranges over *element identifiers* (these unique identifiers are not visualized in the graphical representation of the diagram, but are included in the underlying XML file). We also use a set $\mathbb{E}\mathbb{X}\mathbb{P}$ of *expressions*, ranged over by exp , whose precise syntax is deliberately not specified; we just assume that expressions contain message parameters and values. Such design choice has been influenced by the fact that the expression language operating on data is left unspecified even by the BPMN standard.

Notably, we are not proposing a new modeling formalism, but we are only using a textual notation for the BPMN elements. With respect to the graphical notation, the textual one is more manageable for supporting the formal definition of the semantics and its implementation. The one-to-one correspondence between the graphical notation of BPMN and the syntax used here is shown in Fig 2 (due to lack of space, we omit the case of the event-based gateway). We will only consider terms of the textual syntax that are derived from BPMN models.

$ \begin{aligned} C ::= & \text{start}(\text{id}, e_0, e_1) \mid \text{end}(\text{id}, e) \mid \text{andSplit}(\text{id}, e, E) \\ & \mid \text{andJoin}(\text{id}, E, e) \mid \text{xorSplit}(\text{id}, e, X) \mid \text{xorJoin}(\text{id}, E, e) \\ & \mid \text{task}(\text{id}, e_1, p_1, m, p_2, e_2) \mid \text{eventBased}(\text{id}, e, T) \mid C_1 \mid C_2 \\ X ::= & (e, \text{exp}) \mid X_1, X_2 \quad T ::= (p_1, m, p_2, e) \mid T_1, T_2 \end{aligned} $
--

Fig. 1. Syntax of BPMN Choreography Structures.

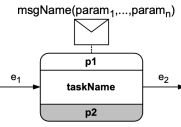
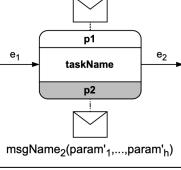
Graphical notation	Textual representation	Graphical notation	Textual representation
	$\text{start}(\text{id}, e_0, e)$		$\text{end}(\text{id}, e)$
	$\text{andSplit}(\text{id}, e, \{e_1, \dots, e_n\})$		$\text{andJoin}(\text{id}, \{e_1, \dots, e_n\}, e)$
	$\text{xorSplit}(\text{id}, e, \{(e_1, \text{exp}_1), \dots, (e_n, \text{exp}_n)\})$		$\text{xorJoin}(\text{id}, \{e_1, \dots, e_n\}, e)$
	$\text{task}(\text{taskName}, e_1, p_1, \text{msgName}(\text{param}_1, \dots, \text{param}_n), p_2, e_2)$		$\text{task}(\text{taskName}, e_1, p_1, \text{msgName}_1(\text{param}_1, \dots, \text{param}_n), p_2, e)$ $\mid$ $\text{task}(\text{taskNameResp}, e, p_2, \text{msgName}_2(\text{param}'_1, \dots, \text{param}'_h), p_1, e_2)$

Fig. 2. Correspondence between BPMN graphical notation and our textual notation.

Notice that two-way tasks are rendered in our formal framework as pairs of one-way tasks, hence they are not explicitly included in the syntax.

To achieve a *compositional definition*, each sequence edge of the BPMN model is split in two parts: the part is outgoing from the source element, and the part is incoming into the target element. The two parts are correlated by means of unique sequence edge names in the BPMN model.

Example 1. Let us consider the BPMN choreography model in Fig. 3. The textual representation of its structure is as follows (for reader's convenience, we use e_i , with i a natural number, to denote sequence edges, m_{msgName} to denote messages, and p_p, p_r, p_w to denote the three participants):

```

start(start1, e0, e1)
| task(takeAppointment, e1, p_p, m_appointment, p_r, e2)
| xorJoin(xor1, {e2, e6}, e3)

```

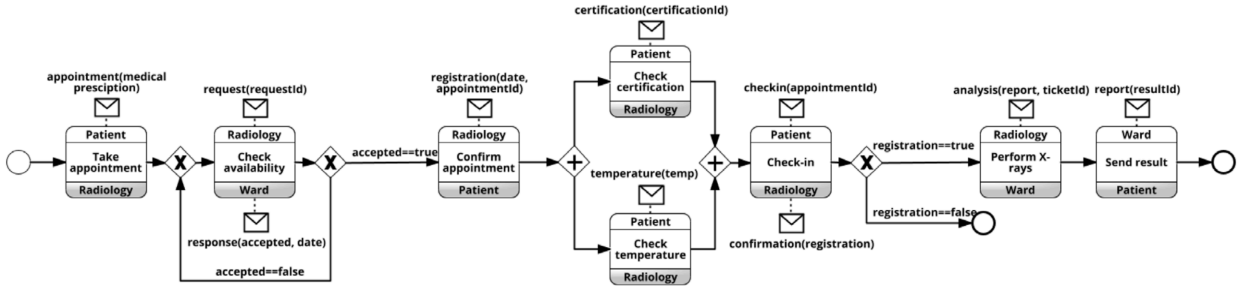


Fig. 3. X-rays exam choreography.

```

| task(checkAvailability, e3, pr, mrequest, pw, e4)
| task(checkAvailabilityResp, e4, pw, mresponse, pr, e5)
| xorSplit(xor2, e5, {(e6, response.accepted == false),
                    (e7, response.accepted == true)})
| task(confirmAppointment, e7, pr, mregistration, pp, e8)
| andSplit(and1, e8, {e9, e10})
| task(checkCertification, e9, pp, mcertification, pr, e11)
| task(checkTemperature, e10, pp, mtemperature, pr, e12)
| andJoin(and2, {e11, e12}, e13)
| task(checkIn, e13, pp, mcheckin, pr, e14)
| task(checkInResp, e14, pr, mconfirmation, pp, e15)
| xorSplit(xor3, e15, {(e16, confirmation.registration == true),
                    (e17, confirmation.registration == false)})
| end(end1, e17)
| task(performXRays, e16, pr, manalysis, pw, e18)
| task(sendResult, e18, pw, mreport, pp, e19)
| end(end2, e19)

```

The operational semantics we propose is given in terms of configurations of the form $\langle C, \sigma, \gamma \rangle$, where C is a choreography structure, $\sigma : \mathbb{E} \rightarrow \mathbb{N}$ is an *execution state function* mapping sequence edges to numbers of tokens ($\mathbb{N}$ is the set of natural numbers), and $\gamma : \mathbb{M} \rightarrow \mathbb{V}$ is a *message state function* mapping messages to values. The execution state obtained by updating in the state σ the number of tokens of the edge e to n , written as $\sigma \cdot \{e \mapsto n\}$, is defined as follows: $(\sigma \cdot \{e \mapsto n\})(e')$ returns n if $e' = e$, otherwise it returns $\sigma(e')$. The message state obtained by updating in the state γ the value of a message m to a value v , written as $\gamma \cdot \{m \mapsto v\}$, is defined similarly. The *initial* states of a choreography, where all sequence edges are unmarked (but for the spurious ones that are marked by one token) and all message values are undefined, are denoted respectively by σ_0 and γ_0 .

The operational semantics is defined by means of a labelled transition system (LTS) on choreography configurations, formalising the execution of a choreography in terms of marking evolution and message exchanges. The LTS is a triple $\langle \mathcal{C}, \mathcal{L}, \rightarrow \rangle$ where: $\mathcal{C}$ is the set of choreography configurations; $\mathcal{L}$, ranged over by l , is the set of *labels* (of transitions that choreography configurations can per-

form); and $\rightarrow \subseteq \mathcal{C} \times \mathcal{L} \times \mathcal{C}$ is the *transition relation*. As usual, we will write $\langle C, \sigma, \gamma \rangle \xrightarrow{l} \langle C, \sigma', \gamma' \rangle$ to indicate that $(\langle C, \sigma, \gamma \rangle, l, \langle C, \sigma', \gamma' \rangle) \in \rightarrow$ and say that the choreography configuration $\langle C, \sigma, \gamma \rangle$ performs a transition labelled by l and becomes the configuration $\langle C, \sigma', \gamma' \rangle$. Since choreography execution only affects the current states, and not the choreography structure, for the sake of presentation, we omit the structure from the target configuration of the transition. Thus, a transition $\langle C, \sigma, \gamma \rangle \xrightarrow{l} \langle C, \sigma', \gamma' \rangle$ is written as $\langle C, \sigma, \gamma \rangle \xrightarrow{l} \langle \sigma', \gamma' \rangle$. Labels l represent computational steps and are defined as: τ , denoting movements of tokens; and $p_1 \rightarrow p_2 : m$, denoting an exchange of message m from participant p_1 to p_2 . Notably, despite the presence of labels, this has to be thought of as a reduction semantics, because labels are not used for synchronisation (as instead it usually happens in labeled semantics), but only for keeping track of the exchanged messages.

The transition relation over choreography configurations is defined by the rules in Fig. 4. Before commenting on the rules, we introduce the auxiliary functions they exploit. Specifically, function $inc : \mathbb{S}_e \times \mathbb{E} \rightarrow \mathbb{S}_e$ (resp. $dec : \mathbb{S}_e \times \mathbb{E} \rightarrow \mathbb{S}_e$), where $\mathbb{S}_e$ is the set of execution states, allows updating a state by incrementing (resp. decrementing) by one the number of tokens marking an edge in the state. Formally, they are defined as follows: $inc(\sigma, e) = \sigma \cdot \{e \mapsto \sigma(e) + 1\}$ and $dec(\sigma, e) = \sigma \cdot \{e \mapsto \sigma(e) - 1\}$. These functions extend in a natural way to sets of edges as follows: $inc(\sigma, \emptyset) = \sigma$ and $inc(\sigma, \{e\} \cup E) = inc(inc(\sigma, e), E)$; the cases for dec are similar. The boolean predicate $eval : \mathbb{E}XP \times \mathbb{S}_m \rightarrow \{true, false\}$, where $\mathbb{S}_m$ is the set of message states, evaluates an expression with respect to a given message state; since the expression language is left unspecified, the definition of $eval$ is unspecified as well. We also use the function $upd : \mathbb{S}_m \times \mathbb{M} \rightarrow \mathbb{S}_m$ that updates a message state by assigning a value to a given message; formally, the function is defined as follows: $upd(\gamma, m) = \gamma \cdot \{m \mapsto v\}$ for $v \in \mathbb{V}$.

We now briefly comment on the operational rules. Rule *Start* starts the execution of a choreography when it is in its initial state, i.e., only the spurious edge is marked, and no message has been exchanged yet. The effect of the rule is to consume the token in the spurious edge and increment the number of tokens in the edge outgoing from the start event. Rule *End*, instead, is enabled when there is at least a token in the incoming edge of the end event, which is then removed. Rule *AndSplit* is applied when there is at least one token in the incoming edge of an AND-Split gateway; as a result of its application the rule decrements the number of tokens in the incoming edge and increments that in each outgoing edge. Rule *AndJoin* decrements the tokens in each incoming edge and increments the number of tokens of the outgoing edge, when each incoming edge has at least one token. Rule *XorSplit* is applied when a token is available in the incoming edge of an XOR split gateway; the rule decrements the token in the incoming edge and increments the tokens in one of the outgoing edges corresponding to a positive evaluation of the associated expression. Rule *XorJoin* is activated every time there is a token in one of the incoming edges, which is then moved to the outgoing edge. Rule *Task* is activated when there is a token in the incoming edge of a choreography task and moves the token from

$\langle \text{start}(\text{id}, \mathbf{e}_0, \mathbf{e}_1), \sigma_0, \gamma_0 \rangle \xrightarrow{\tau} \langle \text{inc}(\text{dec}(\sigma_0, \mathbf{e}_0), \mathbf{e}_1), \gamma_0 \rangle$	$\sigma(\mathbf{e}_0) > 0$	(Start)
$\langle \text{end}(\text{id}, \mathbf{e}), \sigma, \gamma \rangle \xrightarrow{\tau} \langle \text{dec}(\sigma, \mathbf{e}), \gamma \rangle$	$\sigma(\mathbf{e}) > 0$	(End)
$\langle \text{andSplit}(\text{id}, \mathbf{e}, E), \sigma, \gamma \rangle \xrightarrow{\tau} \langle \text{inc}(\text{dec}(\sigma, \mathbf{e}), E), \gamma \rangle$	$\sigma(\mathbf{e}) > 0$	(AndSplit)
$\langle \text{andJoin}(\text{id}, E, \mathbf{e}), \sigma, \gamma \rangle \xrightarrow{\tau} \langle \text{inc}(\text{dec}(\sigma, E), \mathbf{e}), \gamma \rangle$	$\forall \mathbf{e}' \in E . \sigma(\mathbf{e}') > 0$	(AndJoin)
$\langle \text{xorSplit}(\text{id}, \mathbf{e}', \{(\mathbf{e}, \text{exp})\} \cup X), \sigma, \gamma \rangle \xrightarrow{\tau} \langle \text{inc}(\text{dec}(\sigma, \mathbf{e}'), \mathbf{e}), \gamma \rangle$	$\sigma(\mathbf{e}') > 0 \wedge \text{eval}(\text{exp}, \gamma)$	(XorSplit)
$\langle \text{xorJoin}(\text{id}, \{\mathbf{e}\} \cup E, \mathbf{e}'), \sigma, \gamma \rangle \xrightarrow{\tau} \langle \text{inc}(\text{dec}(\sigma, \mathbf{e}), \mathbf{e}'), \gamma \rangle$	$\sigma(\mathbf{e}) > 0$	(XorJoin)
$\langle \text{task}(\text{id}, \mathbf{e}_1, \mathbf{p}_1, \mathbf{m}, \mathbf{p}_2, \mathbf{e}_2), \sigma, \gamma \rangle \xrightarrow{\mathbf{p}_1 \rightarrow \mathbf{p}_2 : \mathbf{m}} \langle \text{inc}(\text{dec}(\sigma, \mathbf{e}_1), \mathbf{e}_2), \text{upd}(\gamma, \mathbf{m}) \rangle$	$\sigma(\mathbf{e}_1) > 0$	(Task)
$\langle \text{eventBased}(\text{id}, \mathbf{e}', \{(\mathbf{p}_1, \mathbf{m}, \mathbf{p}_2, \mathbf{e})\} \cup T), \sigma, \gamma \rangle \xrightarrow{\mathbf{p}_1 \rightarrow \mathbf{p}_2 : \mathbf{m}} \langle \text{inc}(\text{dec}(\sigma, \mathbf{e}'), \mathbf{e}), \text{upd}(\gamma, \mathbf{m}) \rangle$	$\sigma(\mathbf{e}') > 0$	(EvtBased)
$\frac{\langle C_1, \sigma, \gamma \rangle \xrightarrow{l} \langle \sigma', \gamma' \rangle}{\langle C_1 C_2, \sigma, \gamma \rangle \xrightarrow{l} \langle \sigma', \gamma' \rangle} (Int_1) \qquad \frac{\langle C_2, \sigma, \gamma \rangle \xrightarrow{l} \langle \sigma', \gamma' \rangle}{\langle C_1 C_2, \sigma, \gamma \rangle \xrightarrow{l} \langle \sigma', \gamma' \rangle} (Int_2)$		

Fig. 4. Semantics of BPMN Choreographies.

the incoming edge to the outgoing one. The rule produces a label describing the message exchange. Moreover, the message state is updated with a new value for the involved message name; we abstract from the computation of the message value, which is indeed non-deterministically selected via the *upd* function. Rule *EvtBased* is enabled each time there is a token in the incoming edge of an event-based gateway, which is moved to the activated outgoing edge. A label describing the message exchange is produced and the corresponding value is updated in the message state. Finally, rules *Int₁* and *Int₂* deal with interleaving.

References

1. Corradini, F., Morichetta, A., Polini, A., Re, B., Tiezzi, F.: Collaboration vs. choreography conformance in BPMN. *Log. Methods Comput. Sci.* **16**(4) (2020)
2. Corradini, F., Morichetta, A., Re, B., Tiezzi, F.: Walking through the semantics of exclusive and event-based gateways in BPMN choreographies. In: *The Art of Modelling Computational Systems. LNCS*, vol. 11760, pp. 163–181. Springer (2019)